BMW Business Analysis Project

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

#synthetic BMW dataset
models = ['BMW 320i', 'BMW X5', 'BMW X3', 'BMW M3', 'BMW 520d', 'BMW 730Li', 'BMW Z4', 'BMW X1']
transmissions = ['Automatic', 'Manual']
fuel_types = ['Petrol', 'Diesel', 'Hybrid', 'Electric']
cities = ['New York', 'Berlin', 'Tokyo', 'Paris', 'Dubai', 'Sydney', 'Toronto', 'Singapore']

n = 5000
data = {
    'model': np.random.choice(models, n),
    'year': np.random.randint(2005, 2024, n),
    'transmission': np.random.choice(transmissions, n),
    'mileage': np.random.randint(5000, 200000, n),
    'fuel_type': np.random.choice(fuel_types, n),
    'engine_size': np.round(np.random.uniform(1.5, 5.0, n), 1),
    'price': np.random.randint(15000, 120000, n),
    'city': np.random.choice(cities, n)
}
df = pd.DataFrame(data)
df['log_price'] = np.log10(df['price'])
df.head()
```

	model	year	transmission	mileage	fuel_type	engine_size	price	city	log_price
0	BMW Z4	2023	Automatic	134681	Electric	1.9	27113	Tokyo	4.433178
1	BMW M3	2019	Manual	168998	Diesel	3.9	109285	New York	5.038561
2	BMW 520d	2019	Automatic	119300	Electric	1.6	117839	Tokyo	5.071289
3	BMW Z4	2008	Automatic	42997	Diesel	3.7	53639	Berlin	4.729481
4	BMW X3	2011	Manual	196012	Diesel	2.0	79036	Berlin	4.897825

Data Exploration

```
df.info()
df.describe()
df.shape
df.dtypes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   model         5000 non-null   object
 1   year          5000 non-null   int64
 2   transmission  5000 non-null   object
 3   mileage       5000 non-null   int64
 4   fuel_type     5000 non-null   object
 5   engine_size   5000 non-null   float64
 6   price         5000 non-null   int64
 7   city          5000 non-null   object
 8   log_price     5000 non-null   float64
dtypes: float64(2), int64(3), object(4)
memory usage: 351.7+ KB
```

	0
model	object
year	int64
transmission	object
mileage	int64
fuel_type	object
engine_size	float64
price	int64
city	object
log_price	float64

dtype: object

Data Selection and Filtering

```
df.loc[0]
df.iloc[0,0]
filtered_df = df.query('mileage < 50000 and price > 50000')
filtered_df.head()
```

	model	year	transmission	mileage	fuel_type	engine_size	price	city	log_price
3	BMW Z4	2008	Automatic	42997	Diesel	3.7	53639	Berlin	4.729481
11	BMW Z4	2021	Manual	7526	Diesel	3.3	62937	Dubai	4.798906
14	BMW X1	2005	Automatic	11574	Petrol	1.8	89229	New York	4.950506
15	BMW 520d	2012	Automatic	13733	Electric	2.9	58831	Singapore	4.769606
18	BMW X1	2007	Automatic	41051	Hybrid	2.8	82542	Berlin	4.916675

Data Manipulation

```
df_dropped = df.drop(columns=['engine_size'])
df_renamed = df.rename(columns={'model':'car_model'})
df_sorted = df.sort_values(by='price')
df_unique = df.drop_duplicates()
df_filled = df.fillna(0)
df_replaced = df.replace({'Automatic':'Auto'})
df_replaced.head()
```

	model	year	transmission	mileage	fuel_type	engine_size	price	city	log_price
0	BMW Z4	2023	Auto	134681	Electric	1.9	27113	Tokyo	4.433178
1	BMW M3	2019	Manual	168998	Diesel	3.9	109285	New York	5.038561
2	BMW 520d	2019	Auto	119300	Electric	1.6	117839	Tokyo	5.071289
3	BMW Z4	2008	Auto	42997	Diesel	3.7	53639	Berlin	4.729481
4	BMW X3	2011	Manual	196012	Diesel	2.0	79036	Berlin	4.897825

Grouping and Aggregation

```
grouped_city = df.groupby('city')['price'].mean()
agg_model = df.groupby('model').agg({'mileage':'mean', 'price':'mean'})
count_model = df.groupby('fuel_type')['model'].count()
grouped_city, agg_model.head(), count_model
```

```
(city
 Berlin       66267.533871
 Dubai        67686.247634
 New York     67458.312397
 Paris        68084.320724
 Singapore    65323.108626
 Sydney       70342.147059
 Tokyo        67531.717949
 Toronto      65871.238924
 Name: price, dtype: float64,
                  mileage         price
 model
 BMW 320i   100499.124224  68190.408385
 BMW 520d   100820.969278  66529.725038
 BMW 730Li  100813.918790  68787.434713
 BMW M3     104263.980213  66864.684932
 BMW X1     106248.368948  68270.170284,
 fuel_type
 Diesel      1266
 Electric    1253
 Hybrid      1250
 Petrol      1231
 Name: model, dtype: int64)
```

String Operations

```
df['contains_X'] = df['model'].str.contains('X')
df['model'] = df['model'].str.strip()
df.head()
```

	model	year	transmission	mileage	fuel_type	engine_size	price	city	log_price	contains_X
0	BMW Z4	2023	Automatic	134681	Electric	1.9	27113	Tokyo	4.433178	False
1	BMW M3	2019	Manual	168998	Diesel	3.9	109285	New York	5.038561	False
2	BMW 520d	2019	Automatic	119300	Electric	1.6	117839	Tokyo	5.071289	False
3	BMW Z4	2008	Automatic	42997	Diesel	3.7	53639	Berlin	4.729481	False

Statistical Analysis

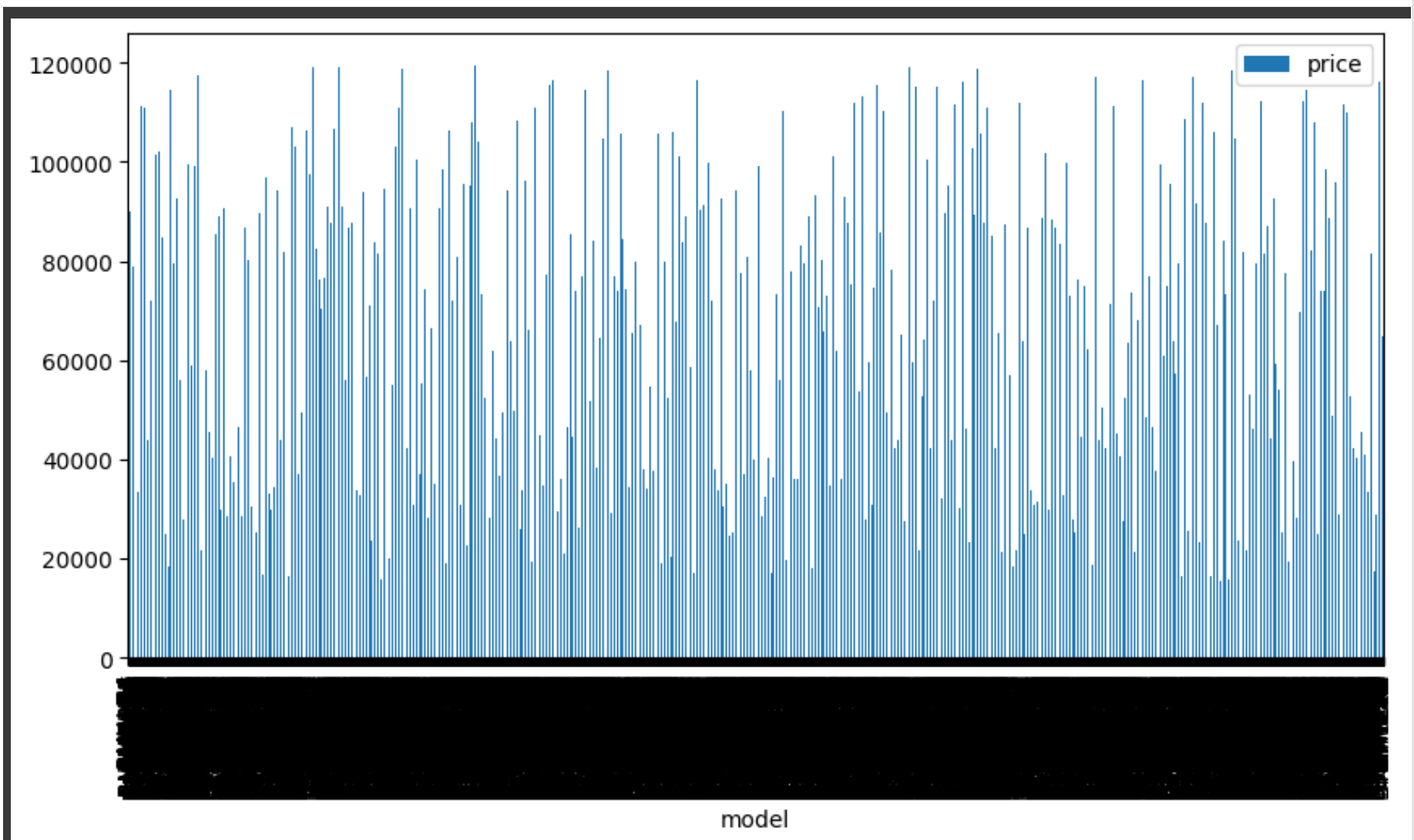
```
df.corr(numeric_only=True)
df['fuel_type'].value_counts()
```

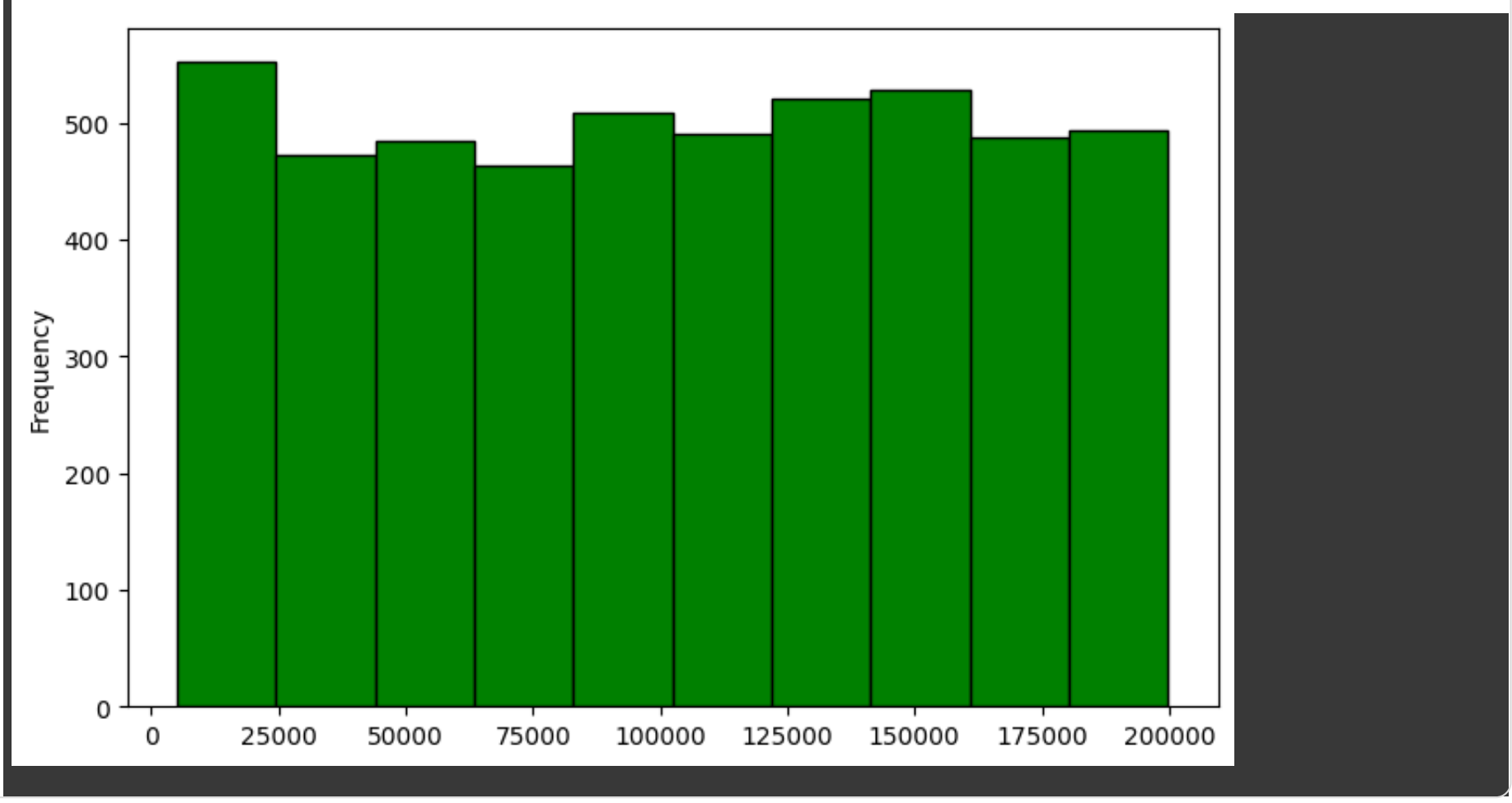
fuel_type	count
Diesel	1266
Electric	1253
Hybrid	1250
Petrol	1231

dtype: int64

Visualization

```
df.plot(x='model', y='price', kind='bar', figsize=(10,5))
plt.show()
df['mileage'].plot(kind='hist', color='green', edgecolor='black', figsize=(8,5))
plt.show()
```

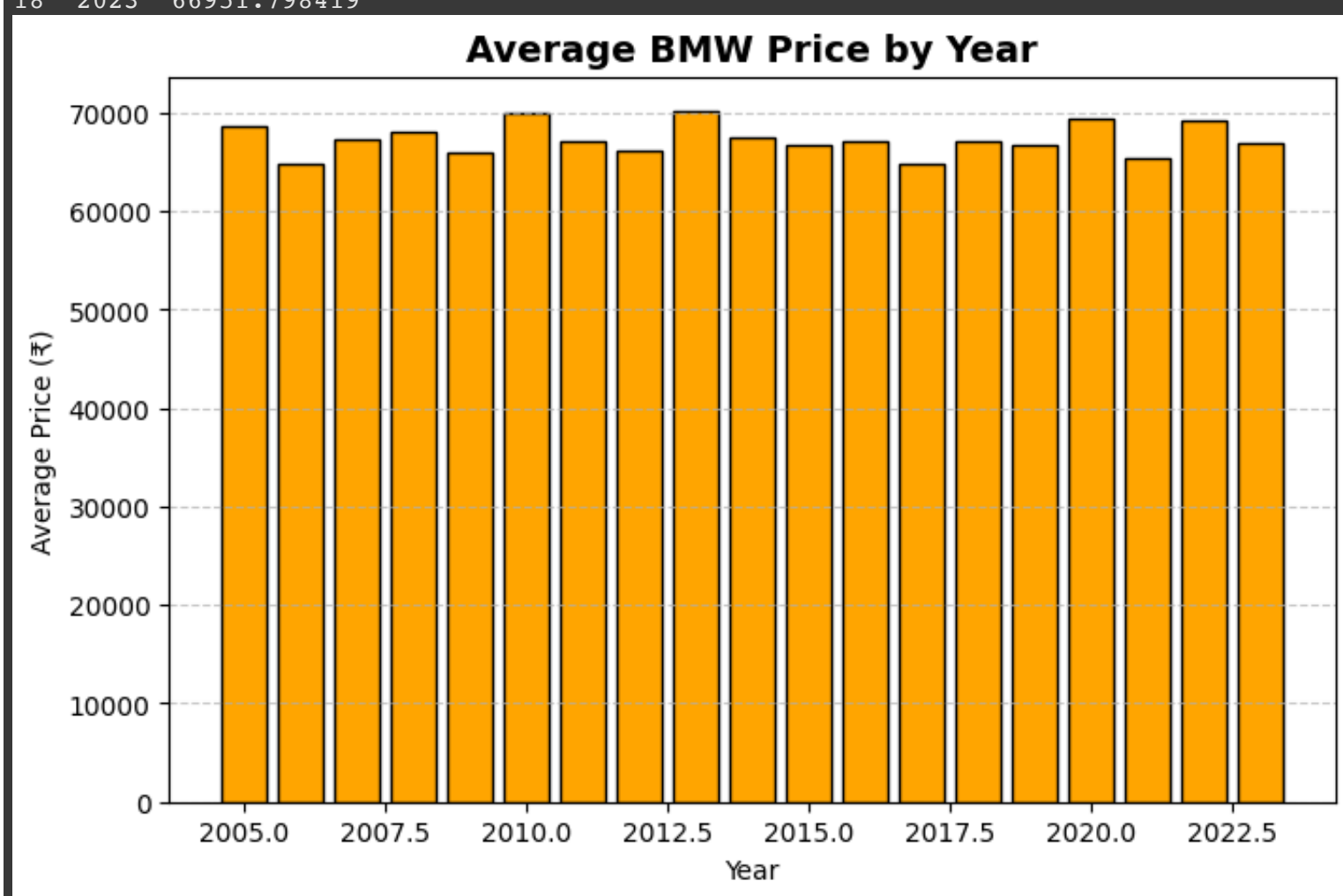




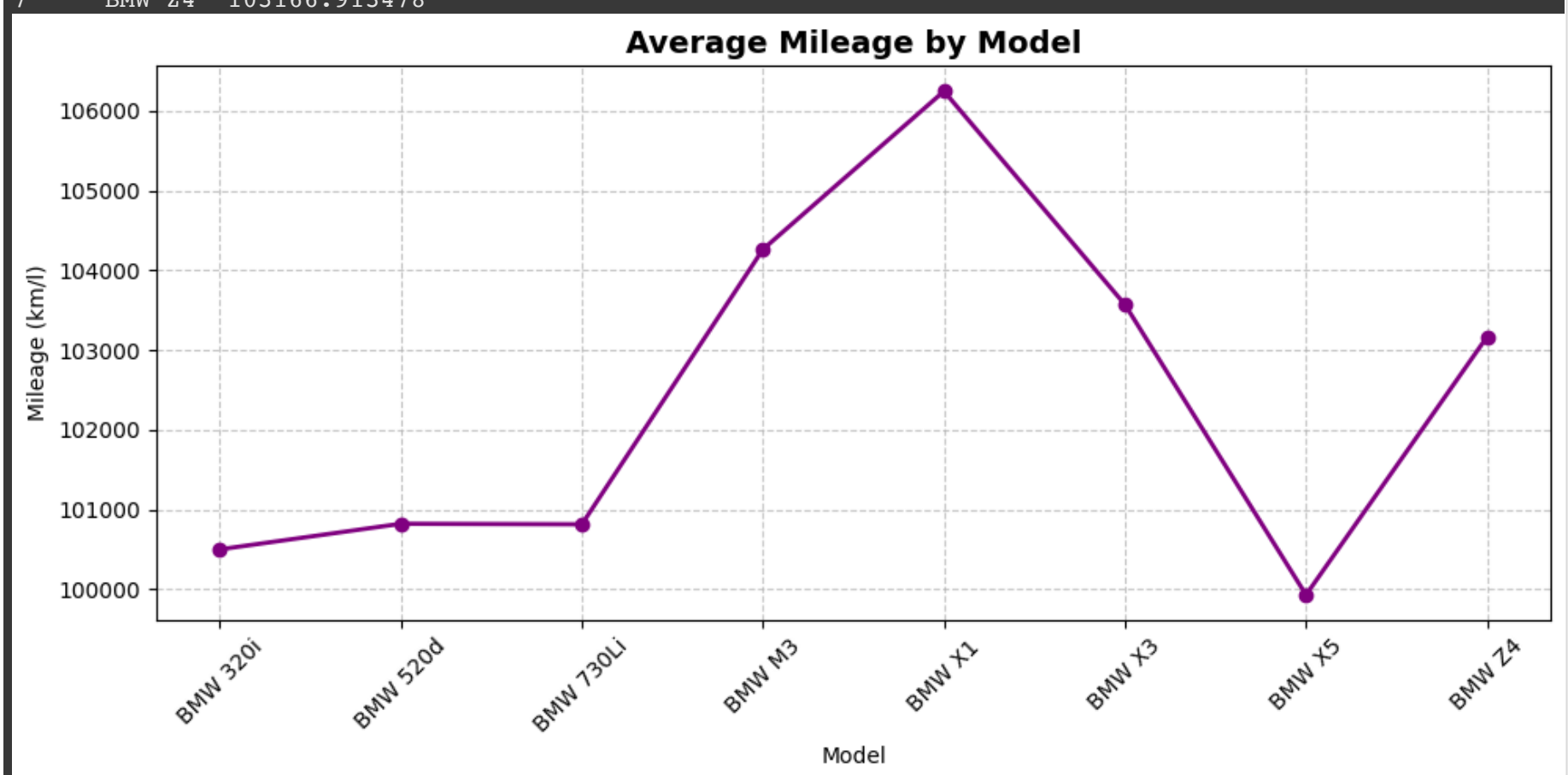
```
 #Average price by year
avg_price_year = df.groupby('year')['price'].mean().reset_index()
print("Average Price by Year:\n", avg_price_year)

plt.figure(figsize=(8,5))
plt.bar(avg_price_year['year'], avg_price_year['price'], color='orange', edgecolor='black')
plt.title('Average BMW Price by Year', fontsize=14, fontweight='bold')
plt.xlabel('Year')
plt.ylabel('Average Price (₹)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

```
Average Price by Year:
     year         price
0   2005  68660.861925
1   2006  64821.625468
2   2007  67249.342412
3   2008  68048.621094
4   2009  66072.976285
5   2010  70070.961832
6   2011  67153.329670
7   2012  66256.671875
8   2013  70179.042373
9   2014  67482.738351
10  2015  66713.915385
11  2016  67140.256604
12  2017  64866.092150
13  2018  67176.882353
14  2019  66738.137037
15  2020  69451.916968
16  2021  65400.934932
17  2022  69149.259124
18  2023  66951.798419
```



```
# Average mileage by model
avg_mileage_model = df.groupby('model')['mileage'].mean().reset_index()
print("Average Mileage by Model:\n", avg_mileage_model)

plt.figure(figsize=(10,5))
plt.plot(avg_mileage_model['model'], avg_mileage_model['mileage'], marker='o', color='purple', linewic
plt.title('Average Mileage by Model', fontsize=14, fontweight='bold')
plt.xlabel('Model')
plt.ylabel('Mileage (km/l)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

```
Average Mileage by Model:
        model        mileage
0   BMW 320i  100499.124224
1   BMW 520d  100820.969278
2  BMW 730Li  100813.918790
3     BMW M3  104263.980213
4     BMW X1  106248.368948
5     BMW X3  103569.835526
6     BMW X5   99928.691176
7     BMW Z4  103166.913478
```